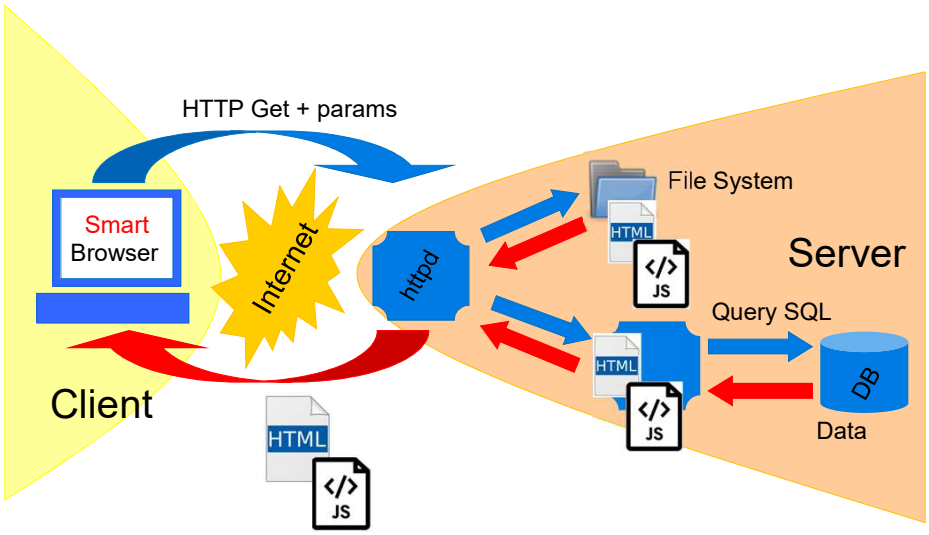
UNIVERSITÀ DI TRENTO

Javascript : the basis of the language

Execute code also on the client



The web programmer also writes programs which run on the browser
Which language?

JavaScript History

- JavaScript was born as Mocha, then "**LiveScript**" at the beginning of the 94s.
- Name changed into JavaScript (name owned by Netscape)
- Microsoft responds with **Vbscript**
- Microsoft introduces **JScript** (dialect of Javascript)
- A standard is defined: ECMAScript (ECMA-262, ISO-16262)
- Jscript survives (as ECMAScript incarnation till 2009, then as Chakra till 2015)
- Another incarnation of ECMAScript is **ActionScript** (Adobe, for Flash)

Examples of what JavaScript can do

- Changing HTML content
- Changing HTML attribute values
- Changing HTML styles (CSS)
- Hiding HTML elements
- Displaying HTML elements
- …

The dynamic behaviour is on the client side!
(the file can be loaded locally)

JavaScript is event-based!

Changing HTML content

```
<!DOCTYPE html>
<html>
<body>

<h2>What Can JavaScript Do #1?</h2>

<p id="demo">JavaScript can change me!</p>

<button type="button"
onclick="document.getElementById('demo').innerHTML =
'Hey! I have changed!'">Click here to change!</button>

</body>
</html>
```

Changing HTML style

```
<!DOCTYPE html>
<html>
    <head>
        <title>What can JavaScript do #2</title>
    </head>
    <body>
        <div onmouseover="this.style.color = 'red'"
            onmouseout="this.style.color = 'green'">
            I can change my colour!</div>
    </body>
</html>
```

JavaScript and HTML

There are three way through which users can use JavaScript in HTML:

- Embedding code
- Inline code
- External file

JavaScript and HTML

- Inline code (in an event handler):
  you add JavaScript directly in the html without using <script>
- Embedding code
  You use the <script>…</script> tag (you can also define JavaScript code in the <body> tag or the <head> tag)
- External file:
  you create a file to hold the code of JavaScript with the .js extension. Then, you incorporate/include it into our HTML document using the src attribute of <script>.
  It is helpful if to use the same code in several HTML documents

Base

Syntax is C-like (C++-like, Java-like)
case-sensitive,
statements end with (optional) semicolon ;
//comment /*comment*/
operators (=,*,+,++,+=,!=,==,&&,…)

Conditional Statements

```
if (expression) {statements} else {statements}

switch (expression) {
        case value: statements; break;
        …
        default: statements; break;}
```

Loops

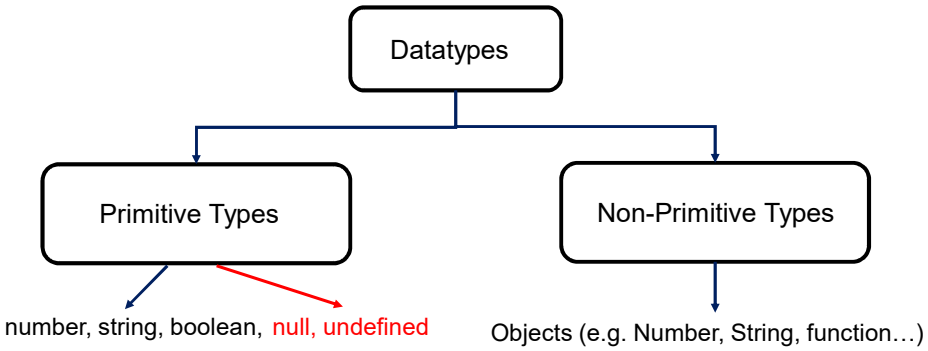
```
while (expression) {statements}

do (expression) while {statements}

for (initialize ; test ; increment) {statements}
```

Some news on the for loop, we will see..

Datatypes



JavaScript is loosely and dynamically typed

Datatypes

```
t0 = typeof(x);
var x = 3;
var t1 = typeof(x);
x = "pippo";
var t2 = typeof(x);
```

t0: undefined

t1: number

t2: string

Operators

- Mathematical operators: standard, plus ** for exponentiation (ES6)
- Assignment operators: standard, plus **= for exponentiation (ES6)
- String operators: + (concatenation), see later
- Comparison operators: standard, plus type and value:

===	equal value and equal type
!==	not equal value or not equal type

- Logical and bitwise operators: standard
- Type operators

typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

How can I do basic I/O in JavaScript?

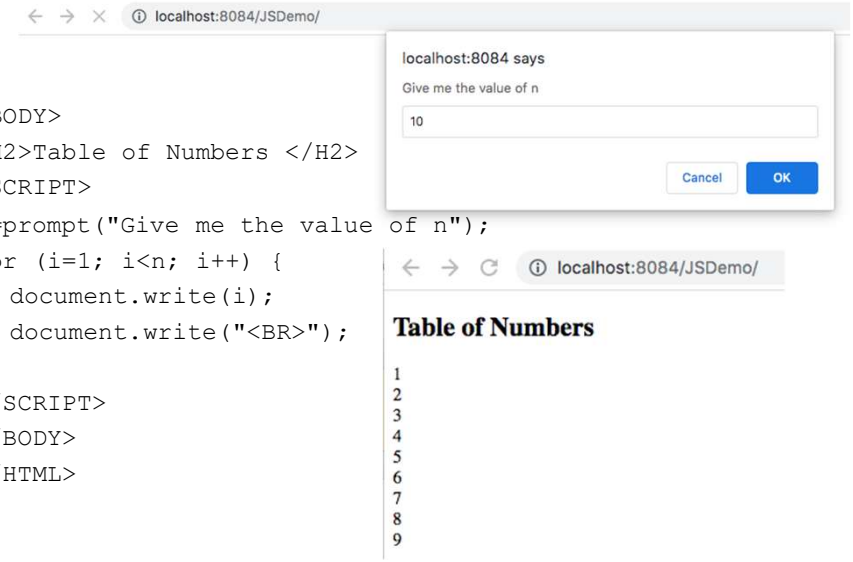
13

JavaScript input

- prompt() instructs the browser to display a dialog with an optional message prompting the user to input some text, and to wait until the user either submits the text or cancels the dialog
- It returns the input value (as a string) if the user clicks "OK", otherwise it returns null

Asking users for inputs

```
<BODY>
<H2>Table of Numbers </H2>
<SCRIPT>
n=prompt("Give me the value of n");
for (i=1; i<n; i++) {
   document.write(i);
   document.write("<BR>");
}
</SCRIPT>
</BODY>
</HTML>
```



JavaScript output

1. Writing into the HTML output using **document.write()**
2. Writing into an alert box, using **alert()**
3. Writing into the browser console, using **console.log()**
4. Writing into an HTML element, using **innerHTML**

document.write

Writing into the HTML output using **document.write()**

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page using JS</h2>
<p>I used document.write</p>

<script>
document.write(5 + 6);
</script>

</body>
</html>
```


```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page</h1>
<p>My first paragraph.</p>
<button type="button" onclick="document.write(5 + 6)">Try
it</button>

</body>
</html>
```

alert

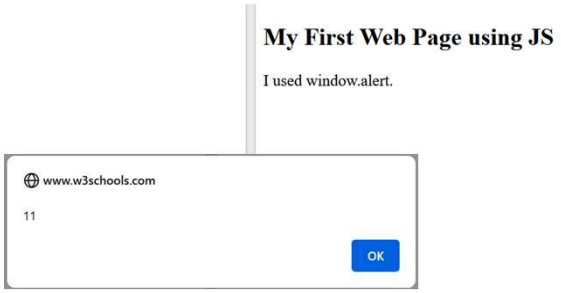
Writing into an alert box, using **alert()**

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page using JS</h2>
<p>I used window.alert.</p>

<script>
window.alert(5 + 6);
</script>

</body>
</html>
```



console.log

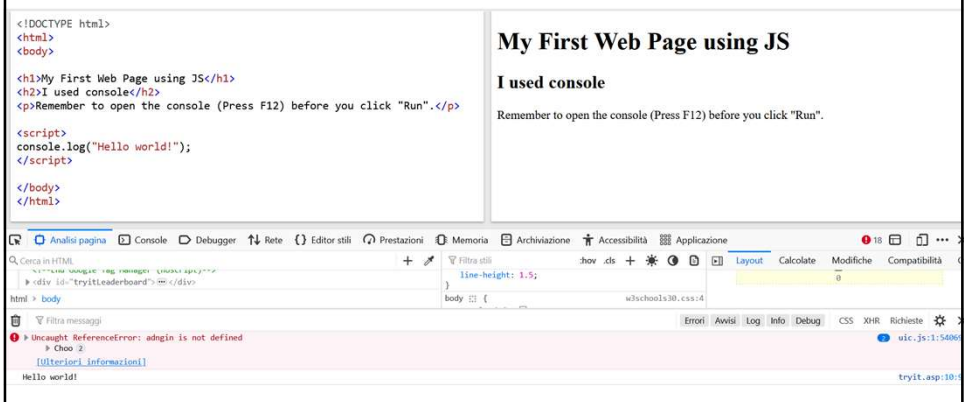
Writing into the browser console, using **console.log()**

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page using JS</h1>
<h2>I used console</h2>
<p>Remember to open the console (Press F12) before you click "Run".</p>

<script>
console.log("Hello world!");
</script>

</body>
</html>
```



innerHTML

- JavaScript can use the document.getElementById(id) method to access an HTML element
- The id attribute defines the HTML element
- The innerHTML property defines the HTML content:

```
<p id ="demo">JavaScript can change me!</p>

<button type="button"
onclick="document.getElementById('demo').innerHTML = 'Hey! I have
changed!'">Click here to change!</button>
```

Pay attention to string and String!

22

Strings

- JavaScript Strings are a double-headed beast…
- They are both:
  - a primitive data type (string) and
  - an object (String)

```
var firstName_primitive = "John";
var firstName_object = new String("John");
```

When using === , firstName_primitive and firstName_object are NOT EQUAL, because the === operator expects equality in both type and value. With == they are EQUAL

String methods

- There are a lot of java-like methods
- Some examples:

  length
  concat(value,…)
  slice(start, end)
  toLowerCase(), toUpperCase()
  charAt(0)
  indexOf(substring), lastIndexOf(substring)
  charCodeAt(n), fromCharCode(value,…)
  replace(regexp, string), search(regexp)

How is the + operator behaving in JS?

25

Operating on Strings

```
a="foot";
b="ball";

a>b => true
```

```
a+b => football
```

+ as a binary operator : rules

The operator "+" is overloaded for two different operations: numeric addition and concatenation of strings.

When a "+" is evaluated:

- **First**, the operands are converted to their primitive data
- **Then**, the types of operands are checked:
  - If an operand is a string, the other operand is converted to a string and they are concatenated
  - Otherwise, both the operands are converted to numbers, and numeric addition is performed:

boolean => 0 (false),1(true)
null => 0
undefined => NaN

+ as a binary operator: examples

x	y	x+y
1	2	3
"1"	2	12
1	null	1
"1"	null	1null
1	undefined	NaN
"1"	undefined	1undefined
1	true	2
"1"	true	1true
false	true	1
true	null	TRY!
true	undefined	
null	null	
null	undefined	

What does happen with more operands?

```
document.write(3+2+"ciao"+3+2)
```

What does happen with more operands?

```
document.write(3+2+"ciao"+3+2)
```

- Think about precedence of operators and associativity rules:
  - If operators have different precedence levels, the operator with the higher *precedence* goes first and associativity does not matter
  - Within operators of the same precedence:
    - right associativity (right-to-left)
    - left associativity (left-to-tight)

Operations with integers

```
<script>
x = "100";
y = "10";
document.write(x * y);document.write("<br>");
document.write(x + y);document.write("<br>");
document.write(x * 1 + y);document.write("<br>");
document.write(x * 1 + y * 1);document.write("<br>");
</script>
```

OUTPUT:
1000
10010
10010
110

+ as unary operator

Unary “+” can be used to convert string to number

```
y = "5";        // y is a string
x = + y;        // x is a number (5)

y = "Pippo";    // y is a string
x = + y;        // x is a number (NaN)
```

Something about functions

33

Functions

```
function f(x) {return x*x}
```

```
<script>
function add(x,y) {return x+y;};
function multiply(x,y) {return x*y;};
function operate(op,x,y) {
                    return op(x,y);};
document.write(operate(add,3,2));
</script>
```

Functions

JavaScript functions:

- do not specify data types for parameters / return value
- do not perform type checking on the passed arguments
- do not check the number of arguments received

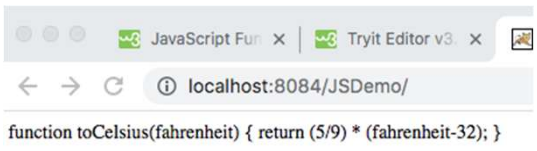
If a function is called with **missing arguments** (less than declared), the missing values are set to undefined

```
add(3) -> ?
```

Note:
EcmaScript 2015 allows default parameter values in the declaration:
function f(x, y = 2) { // …}

Functions

```
<!DOCTYPE html>
<html>
<body>
<script>
function toCelsius(fahrenheit)
{
    return (5/9) * (fahrenheit-32);
}
document.write(toCelsius);
</script>
</body>
</html>
```



Function statements

```
hoisted();

function hoisted()
{
  console.log('This function has been hoisted.');
};
```

OUTPUT:
This function has been hoisted

Function declaration are hoisted

Function hoisting

Hoisting is a JavaScript mechanism where variables and function declarations are moved to the top of their scope before code execution

The hoisting mechanism only moves the declaration. The assignments are left in place

Functions declarations are hoisted first, then variables are hoisted

Function expressions

A JavaScript function can also be defined using an **expression**. A function expression can be stored in a variable:

```
var x = function (a, b) {return a * b};
```

After a function expression has been stored in a variable, the variable can be used as a function: product=x(2,3);

Functions stored in variables do not need function names, as they are always invoked (called) using the variable name.

```
var fundef = function() {
  document.write('Hello');
};

fundef();
```

OUTPUT:
Hello

Function expressions and hoisting

```
fundef();
```

OUTPUT:
"TypeError: expression is not a function

```
var fundef = function() {
  console.log('This will not work.');
};
```

Function expressions are not hoisted

Arrow functions

```
function multiplyByTwo(num){ return num * 2;}
```

Can be written as:

- const multiplyByTwo=function (num){ return num * 2;}
- const multiplyByTwo= (num) => { return num * 2;}
- const multiplyByTwo= num =>{ return num * 2;}
- const multiplyByTwo= num => num * 2;

Usage (in all cases): `multiplyByTwo(4);`

Mapping and filtering functions

```
<script>
twodArray = [1,2,3,4];
document.write( twodArray);
document.write( "<BR>");
document.write( twodArray.map( num => num * 2) );
</script>
```

OUTPUT:
1,2,3,4
2,4,6,8

```
<script>
twodArray = [1,2,3,4];
document.write( twodArray);
document.write( "<BR>");
document.write( twodArray.filter( num => num % 2 == 0));
</script>
```

OUTPUT:
1,2,3,4
2,4

Your turn!

```
<script>
multiplyByTwo=function(num){ return num * 2;}
document.write(multiplyByTwo(6));
document.write("<BR>");
myArray=[1,2,3];
document.write(myArray.map(multiplyByTwo));
</script>
```

OUTPUT:
12
2,4,6

How is scoping defined in JavaScript, and what is the behavior caused by hoisting?

44

Undeclared variables

A variable that has not been assigned a value is of type undefined (and at execution takes value undefined). A method or statement also returns undefined if the variable that is being evaluated does not have an assigned value. A function returns undefined if a value was not returned.

A ReferenceError is thrown when trying to access a previously undeclared variable

```
<script>
  var t0=typeof(x);
  var x=3;
  var t1=typeof(x);
  x="pippo";
  var t2=typeof(x);
  document.write(t0+", "+t1+", "+t2+"<br>");
  document.write(z);
</script>;
```

OUTPUT:
undefined, number, string

NO OUTPUT:
reference error

Variable scope

Type of declaration	Scope	Note
`x=10; x;`	always global	
`var x=10; var x;`	function scope	(global if external to any function)
`let x=10; let x;`	block scope	ES 6

Variable scope

- Variables declared with var live in their function scope (which, if outside any function, is global)
- Variables declared with let have block scope instead of function scope (ES 6)
- Variables declared without var or let are always global.

Re-declaring variables

- Redeclaring variables with var will not trigger an error and the variable will not lose its value (unless the declaration has an initializer)

```
var carName = "Fiat";
var carName;
```
OK

```
var carName = "Fiat";
var carName = "Ferrari";
```
OK

- Re-declaring variables with let will trigger an error: let declarations cannot be redeclared by any other declaration in the same scope

```
let carName = "Volvo";
let carName;
```
ERROR

const

- Const does not define a constant value but it defines a constant reference to a value
- Variables defined as const:
  - must be assigned a value when they are declared
  - const cannot be redeclared / reassigned
  - have block scope

Hoisting of var

Hoisting is a JavaScript mechanism where variables and function declarations are moved to the top of their scope before code execution.

The hoisting mechanism only moves the declaration. The assignments are left in place.

```
…
…
var x=10;
```



```
var x;
…
…
x=10;
```

Variables declared with var are hoisted WITH a default initialization of undefined

Hoisting of let and const?

```
<!DOCTYPE html>
<html>
<body>
<script>
try{
document.write(y);
document.write(z);
var y=2;
let z=10;}

catch (err) { document.write("ERROR",err.message);}

</script>
</body>
</html>
```

Variables declared with let and const are also hoisted but, unlike var, are not initialized with a default value. An exception will be thrown if a variable declared with let or const is read before it is initialized

Function expressions and hoisting

```
var fundef;
```

```
fundef;
```

```
fundef();
```

```
var fundef = function() {
  console.log('This will not work.');
};
```

Variable scope

```
try {
p2 = (n,s) => document.write(n+":"+s+"<br>");
p1 = (n) => document.write(n+"<br>");
// code here can NOT use carName
       function f() {
           var carName="volvo"
           p1(carName); // code here CAN use carName
       }
       f();
       p1(carName); // code here can NOT use carName
} catch (err) {
    p2("ERROR",err.message);
}
```

OUTPUT:
volvo
REfERROR:carName is not defined

Variable scope - example 1

```
try {
p2 = (n,s) => document.write(n+":"+s+"<br>");
p1 = (n) => document.write(n+"<br>");
function f() {
  a = 20;
  var b = 100;
}
f();
p1(a);
p1("<hr>");
p1(b);
} catch (err) {
    p2("ERROR",err.message);
}
```

OUTPUT:
20

ERROR : b is not defined

Variable scope - example 2

```
try {
p2 = (n,s) => document.write(n+":"+s+"<br>");
p1 = (n) => document.write(n+"<br>");
function f() {
  p1(a);
  a = 20;
  var b = 100;
}
f();
} catch (err) {
    p2("ERROR",err.message);
}
```


OUTPUT:
ERROR: a is not defined

Variable scope - example 3

```
try {
p2 = (n,s) => document.write(n+":"+s+"<br>");
p1 = (n) => document.write(n+"<br>");
function f() {
  p1(b);
  a = 20;
  var b = 100;
}
f();
} catch (err) {
    p2("ERROR",err.message);
}
```


OUTPUT:
undefined

because of hoisting!

Variable scope - example 4

```
try {
p2 = (n,s) => document.write(n+":"+s+"<br>");
p1 = (n) => document.write(n+"<br>");
 p1(b);
function f() {
  a = 20;
  var b = 100;
}
f();
} catch (err) {
    p2("ERROR",err.message);
}
```

OUTPUT:
ERROR: b is not defined

Variable scope - example 5

```
<script>
p2 = (n,s) => document.write(n+": "+s+"<br>");
x=null; // DEF 1
function f() {
  var x = "A"; // DEF 2
  p2(2,"x in f "+x);
  {   var x=1; // DEF 3
      p2(3,"x in inner block in f "+x);
  }
  p2(4,"x in f "+x);
}
p2(1,x);
f();
p2(5,x);
</script>
```

OUTPUT:
1: null
2: x in f A
3: x in inner block in f 1
4: x in f 1
5: null